

EXPERIMENT 8

Student Name: Shagun

UID: 24BDA70214

Branch: AIT-CSE

Section/Group: 24AIT_KRG-G2

Subject Name: Computer Network

Subject Code: 24CSH-337

Experiment 8 – Implement Secure Connection using TCP

Objective

- To study the fundamental concepts of TCP (Transmission Control Protocol) and understand its connection-oriented nature.
- To develop a basic client-server communication system using Python socket programming.
- To establish a TCP-based connection between the server.py and client.py programs.
- To examine how a TCP connection is created, how information is exchanged, and how the connection is properly closed.

Theory

Transmission Control Protocol (TCP) is a transport-layer protocol designed to provide dependable communication between devices connected through a network. It is connection-oriented, meaning that a connection must be established between the communicating devices before any data is exchanged. TCP also ensures that data reaches the destination accurately and in the correct order.

TCP generally operates using a **client-server architecture**. The server first creates a socket and associates it with a specific IP address and port. It then waits for connection requests from clients. The client also creates a socket and uses the server's IP address and port number to request a connection. After the connection is successfully established, the client and server can send and receive data.

To maintain reliable communication, TCP uses several mechanisms, including **sequence numbers, acknowledgements, retransmission of missing data, and flow control**. These mechanisms help ensure that transmitted information reaches the receiving system correctly.

Before actual data exchange begins, TCP establishes a connection through a procedure known as the **three-way handshake**:

1. **SYN:** The client initiates communication by sending a synchronization request to the server.
2. **SYN-ACK:** The server responds by acknowledging the client's request and sending its own synchronization signal.
3. **ACK:** The client acknowledges the server's response, thereby completing the connection setup.

EXPERIMENT 8

Once this process is completed, the connection is ready for reliable data transmission.

Practical Procedure

1. Launch a Python-compatible code editor such as VS Code, PyCharm, or IntelliJ IDEA.
2. Create a project directory for the TCP communication program.
3. Inside the directory, create two Python files named server.py and client.py.
4. Enter the appropriate server-side program in server.py and the client-side program in client.py.
5. Save both Python files.
6. Open two separate terminal windows, or use a split-terminal setup in the code editor.
7. In the first terminal, start the server by executing:
python server.py
8. In the second terminal, launch the client using:
python client.py
9. After the connection is established, type and send a message from the client terminal.
10. Enter a response from the server terminal to demonstrate two-way communication.
11. To terminate the communication session, send the message **"Over"** from the client side.

Code

Server Program — server.py

```
import socket

# Create a TCP socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Assign IP address and port number to the server

server_socket.bind(("127.0.0.1", 5000))

# Start listening for incoming client connections

server_socket.listen(1)

print("Server is running")
```

EXPERIMENT 8

```
print("Waiting for client connection...")
# Accept a client request
client_socket, client_address = server_socket.accept()
print("Client connected:", client_address)
while True:
    # Receive data from the client
    message = client_socket.recv(1024).decode()
    print("Client:", message)
    # Stop communication when the client sends "Over"
    if message == "Over":
        break
    # Take a response from the server user
    reply = input("Enter reply: ")
    # Convert the reply into bytes and send it to the client
    client_socket.send(reply.encode())
# Close both sockets
client_socket.close()
server_socket.close()
print("Connection terminated")
```

Client Program — client.py

```
import socket
# Create a TCP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# Establish a connection with the server
client_socket.connect(("127.0.0.1", 5000))
```

EXPERIMENT 8

```
print("Successfully connected to the server")

while True:

    # Read a message from the client user

    message = input("Enter message: ")

    # Convert the message into bytes and transmit it

    client_socket.send(message.encode())

    # Terminate communication when "Over" is entered

    if message == "Over":

        break

    # Receive the server's response

    reply = client_socket.recv(1024).decode()

    print("Server:", reply)

# Close the client socket

client_socket.close()

print("Connection terminated")
```

Result

<pre>PS D:\University\Year 3\CN\TCP_Socket_Experiment\TCP_Socket_Experiment> python client.py Connected to server Enter message: Hi!!, Shagun Server: You deserve 30/30 Enter message: Thank you :)) Server: np worries, You deserve it Enter message: Many thanks for your generosity Server: You should all get 30/30 in all the CN and ADBMS experiments Enter message: Thank You SO MUCH Server: np worries Enter message: Over Connection closed</pre>	<pre>PS D:\University\Year 3\CN\TCP_Socket_Experiment\TCP_Socket_Experiment> python server.py Waiting for a client... Client: Connected: (Client: connected: ('127.0.0.1', 49371) Client: Hi!!, Shagun Enter reply: You deserve 30/30 Client: Thank you :) Enter reply: np worries, You deserve it Client: Many thanks for your generosity Enter reply: You should all get 30/30 in all the CN and ADBMS experiments Client: Thank YOU SO MUCH Enter reply: no worries</pre>
--	--

Learning Outcome

- Understand the working and characteristics of TCP, including connection establishment, reliable data transfer, and connection termination.
- Develop a TCP client-server application using Python socket programming.
- Analyze the process of establishing a connection and exchanging data between a client and server using TCP.4.